OpenCRE structure (controls, mappings, etc.)

OpenCRE (Open Common Requirement Enumeration) is an OWASP-led catalog and linking platform that unifies security standards, controls, and related artifacts (threats, tests, code, etc.) via *Common Requirements* (CREs).[1] [2] [3]

Below is a concise overview of its structure: how controls live in the model, how they are mapped, and how the data is organized.

Core data objects

OpenCRE's model is built around four main types of entities:[4] [2] [5]

- **Common Requirements (CREs)**
  High-level security "concerns" or practices (e.g., "Ensure inputs are validated," "Securely manage secrets"). Each CRE has a numeric ID, a short description, and tags (e.g., category, maturity stage). CREs are the "hub" that connects standards and artifacts.[3] [4]
- **Controls / standard sections**
  Individual controls or sections from external standards (e.g., NIST 800-53 AC-3, CISA SSDF, ISO 27001, PCI-DSS, OWASP ASVS, SAMM, etc.). Each control entry notes its standard, section ID, title, and text.[2] [5] [3]
- **Artifacts / references**
  Additional resources linked to CREs or controls, such as:
  - Vulnerability IDs (CWE, CAPEC)
  - OWASP Top 10 items
  - Testing guidance
  - Code samples and cheat sheets
  - Tooling (e.g., configuration snippets, rules)[6] [1] [4]
- **Mappings**
  Directed links between:
  - A CRE and a control/section in a standard.
  - A CRE and an artifact (e.g., CWE, OWASP Top 10, code sample).
    These are stored as structured "edges" (from_id → to_id) in the catalog DB.[5] [2] [3]

How controls are structured

Within OpenCRE, each control is effectively an entry from an external standard that is normalized into a small schema:[2] [5]

- Standard name (e.g., "NIST 800-53", "PCI-DSS", "OWASP ASVS")
- Section ID (e.g., "AC-3", "3.1.2", "V1")
- Title and description text
- Optional metadata (e.g., priority, type, maturity)

Controls are *not* the primary "master identifiers"; instead, each control is associated with one or more CREs that express the underlying security concept.[5] [3]

Example:
A NIST 800-53 control on "session locking" and an ASVS control on "session timeout" might both map to the same CRE like "Secure session management," allowing you to see all related requirements from different standards under that single topic.[1] [3]

Mapping structure

Mappings are central to OpenCRE's value. Conceptually the mapping graph looks like:[3] [2] [5]

- CRE → [list of controls]
- CRE → [list of CWEs / CAPECs / OWASP items / code samples / tool configs]

In the database, this is typically represented as:

- `Link` table (or equivalent JSON) entries such as:
  - `from_id = CRE-123`, `to_id = NIST:AC-3`, `type = "standard"`
  - `from_id = CRE-123`, `to_id = CWE-125`, `type = "vulnerability"`

The system also supports bidirectional queries:

- Given a control (e.g., NIST AC-3), find all CREs it maps to and then all related standards and artifacts.[5] [3]
- Given a CRE, list all mapped controls and artifacts across standards.[4] [3]

Catalog and tooling aspects

- **Catalog data**
  The catalog is stored as JSON or a similar structured format, with separate objects for CREs, standards/controls, and mappings. This allows for programmatic querying, diffs, and versioning.[6] [2]
- **Tools and ingestion**
  OpenCRE provides:
  - A Python web app and CLI that load and query the catalog.

- ETL scripts to ingest new standards (parsing control IDs, titles, and bodies) and attach them to existing CREs or create new ones.[7] [2]
- Features like `MyOpenCRE` that let users upload CSVs to propose custom mappings from their internal standards to CREs, then inject those back into the catalog.[7]

Rough mental model (table view)

Entity type	Role in OpenCRE
CRE	High-level security topic; the "hub" node in the graph. [4] [3]
Control/section	Concrete requirement from a standard; attached to one or more CREs. [2] [5]
Artifact (CWE, etc.)	Supporting item (vulnerability, test, code) linked to CREs or controls. [1] [4]
Mapping (link)	Edge between CRE ↔ control or CRE ↔ artifact; drives cross-standard visibility. [2] [5]

If you tell me what you're using OpenCRE for (e.g., building your own control framework, mapping to NIST/ISO 27K, or integrating with a toolchain), I can sketch a more concrete data model or example schema tailored to your use case.

⁂

What makes a text "CRE-relevant"?

What structure does a CRE expect?
What kind of text would be useless?

A text is **"CRE-relevant"** if it meaningfully describes or supports a *security requirement topic* and can be linked to or reused by multiple standards, tools, or practices.[11] [12] [13]

Below is a breakdown of: what "CRE-relevant" means, what structure a CRE entry expects, and what kind of text is usually useless.

1. What makes a text "CRE-relevant"?

A text is CRE-relevant when it:

- **Expresses a clear security requirement or practice**, not just general commentary or fluff.
  - ✅ Good: "All user inputs must be validated and sanitized before use in the application."
  - ✅ Good: "Secrets must be stored encrypted at rest and never hardcoded in source code."[12] [11]
- **Can be shared across multiple standards or tools** without being tied to a single framework.
  - For example, the same CRE concept can underlie NIST 800-53, ASVS, ISO 27001, and internal policies.[14] [11]
- **Connects to concrete artifacts**:

- Standards clauses, test cases, CWEs/CAPECs, code samples, or configuration patterns. [13] [11] [12]

If the text is **actionable, re-usable, and linkable** to other resources, it is CRE-relevant.[11] [12]

2. What structure does a CRE expect?

In OpenCRE, each CRE is a structured entry with a small but consistent schema.[15] [12]

At a minimum, a CRE record expects:

- **Unique CRE identifier**
  - Format like `111-111` or similar; this is the internal key for linking.[15]
- **Human-readable name / title**
  - Short, descriptive phrase such as:
    - "Secure session management"
    - "Input validation and output encoding"
  - Stored in a field like `Name of the CRE`.[12] [15]
- **Description / requirement text**
  - One or a few sentences that clearly state:
    - *What* the security topic is.
    - *Why* it matters.
  - Example:
    - "Validate all user-controlled input against a known-good schema and encode output appropriately to prevent injection attacks."[11] [12]
- **Hierarchy / parent–child relationships (optional but recommended)**
  - A CRE can be a child of a more general CRE (forming a tree).
  - Organized via fields such as `CRE0` (root), `CRE1` (children of root), etc., in CSV-style ingestion.[16] [15]
- **Tags / categories**
  - Words like `authentication`, `cryptography`, `logging`, `supply-chain`, which help group and search CREs.[16] [12]
- **Links to standards / artifacts (via mapping data)**
  - Not in the CRE body itself, but in the mapping layer:
    - "CRE-111-111 → NIST 800-53 AC-3"
    - "CRE-111-111 → CWE-79 (XSS)"
  - These are stored separately in the catalog's mapping tables.[13] [14] [11]

3. What kind of text would be useless (for CREs)?

Text that is **vague, purely decorative, or non-actionable** tends to be useless for a CRE. Typical examples:

- **Very generic or marketing-style prose**
  - ✕ "This is important for security."
  - ✕ "We must follow best practices."
  - These say nothing concrete and cannot be mapped to tests, standards, or code.[16] [12]
- **Purely narrative or motivational text**
  - ✕ Long stories about security breaches with no explicit requirement.
  - ✕ "Security is everyone's responsibility."
  - These are fine for blogs or trainings, but they don't define a security topic a CRE can represent.[12] [16]
- **Text that cannot be reused or linked**
  - ✕ "Implement the recommendations from this document as appropriate."
  - ✕ Very context-specific internal wording that only applies to one project or team.[16]
- **Redundant or cosmetic lines with no information**
  - Some analyses concluded that certain lines such as raw CRE IDs repeated as labels (`CRE-111-111` again as a separate line) add no value and should be removed.[16]

In short, **useless CRE-text is anything that cannot be turned into a concrete requirement, cannot be reused across frameworks, and cannot be linked to standards, tests, or vulnerabilities.**[11] [12] [16]

If you like, you can paste a candidate paragraph and I can annotate whether it is CRE-relevant, how to structure it, and what to remove or rewrite.

⁂

1. https://cloudsecurityalliance.org/blog/2023/09/27/opencre-org-the-how-and-the-why-of-security-best-practices
2. https://github.com/OWASP/OpenCRE
3. https://owaspsamm.org/blog/2023/09/20/owasp-samm-now-connects-to-opencre/
4. https://owasp.org/opencre/projects/2023/09/11/openCRE
5. https://cloudsecurityalliance.org/press-releases/2023/09/28/cloud-controls-matrix-ccm-now-mapped-to-opencre-cloud-security-alliance-announces
6. https://www.youtube.com/watch?v=SPC8NATkxqo
7. https://github.com/OWASP/OpenCRE/issues/585
8. https://www.linkedin.com/posts/ankittejyadav_ai-security-owasp-activity-7384308735680266240-PGHU
9. https://open-control.org/faq/

10. https://www.opencre.org/chatbot
11. https://cloudsecurityalliance.org/blog/2023/09/27/opencre-org-the-how-and-the-why-of-security-best-practices
12. https://devguide.owasp.org/en/03-requirements/03-opencre/
13. https://github.com/OWASP/OpenCRE
14. https://owaspsamm.org/blog/2023/09/20/owasp-samm-now-connects-to-opencre/
15. https://github.com/OWASP/OpenCRE/blob/main/docs/my-opencre-user-guide.md
16. https://www.cs.ru.nl/bachelors-theses/2022/Thomas_Klein_Breteler___4068246___Improving_OpenCRE.pdf
17. https://www.opencre.org/map_analysis?base=ASVS&compare=OWASP+Web+Security+Testing+Guide+(WSTG)
18. https://rightsindevelopment.org/wp-content/uploads/securepdfs/CRE-Design-Final-doc-ENG.pdf
19. https://jsparkes.com/opencre-chatbot/
20. https://github.com/OWASP/OpenCRE/issues/359